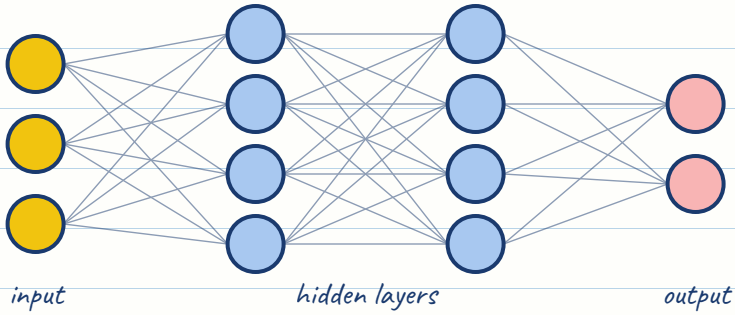
Deep Learning

~ handwritten notes ~



contents:

1. Neurons & MLP · 2. Activations · 3. Backprop
4. Loss & Optimizers · 5. Regularization
6. CNNs · 7. RNNs · 8. Transformers
9. GANs · 10. VAEs · 11. Diffusion · 12. Training Tips

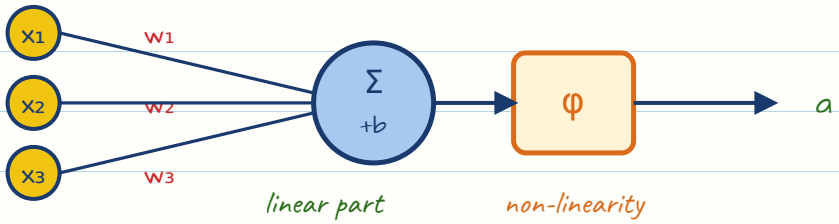
— Sreemanti's notes ✏

1. Neurons & MLP

✦ The Perceptron

A single neuron computes a weighted sum + activation:

$$a = \varphi(w^T x + b)$$



✦ Multi-Layer Perceptron (MLP)

Stack neurons in layers — each layer feeds into the next.

$$h^{(\ell)} = \varphi(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)})$$

With nonlinear φ, an MLP is a universal approximator.

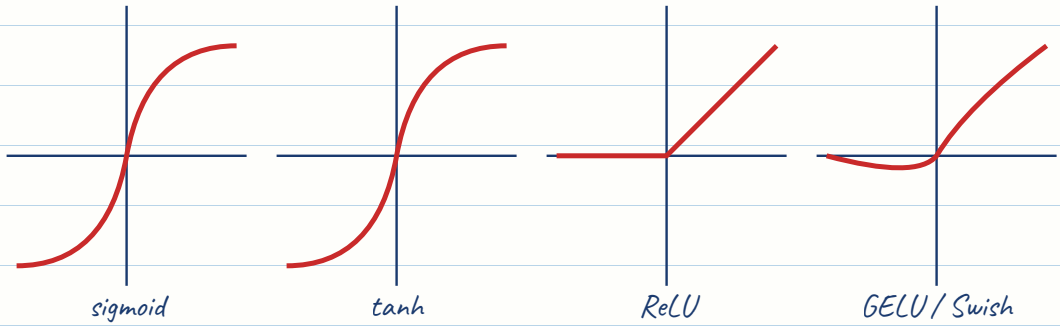
✦ Why "Deep"?

- each layer learns a representation of the input
- early layers → edges, late layers → objects (in vision)
- depth ⇒ exponential representational efficiency vs width
- but → vanishing gradients, optimisation harder ⚠

> Key insight: nonlinear activations are what give DNNs power. Without them, stacking layers = one big linear map.

2. Activation Functions

Inject non-linearity. Without them, the whole net collapses to a linear map.



✦ The Classics

Sigmoid

$\sigma(x) = 1 / (1 + e^{-x}) \in (0,1)$

✗ saturates, kills gradients

Tanh

$\tanh(x) \in (-1, 1)$, zero-centred

✗ still saturates

✦ The Modern Defaults

ReLU

$\max(0, x)$

✓ fast, no saturation for $x > 0$

✗ "dying ReLU" if always negative

Leaky / PReLU

$x < 0$: αx ($\alpha \approx 0.01$)

✓ fixes dying ReLU

GELU

$x \cdot \Phi(x)$, smooth, used in Transformers

Swish / SiLU

$x \cdot \sigma(x)$, self-gated, similar to GELU

✦ Output Layer Activations

- Linear — regression
- Sigmoid — binary classification
- Softmax — multiclass (k-way): $e^{z_i} / \Sigma e^{z_j}$

3. Backpropagation

How do we compute $\partial L/\partial w$ for every weight efficiently? Chain rule + dynamic programming → backprop.

✦ Forward Pass

Compute outputs layer by layer, storing intermediate activations.

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \varphi(z^{(\ell)})$$

✦ Backward Pass

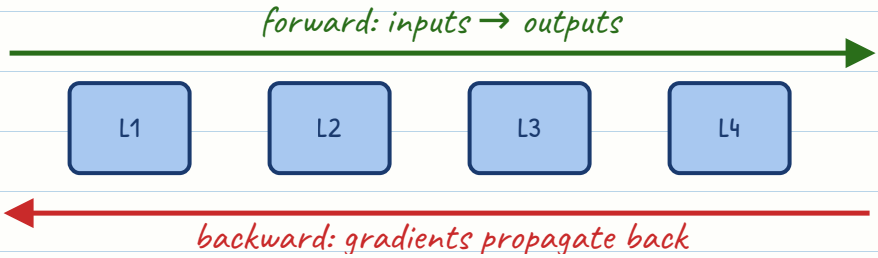
Start at the loss, propagate gradients backwards:

$$\delta^{(L)} = \nabla_a L \odot \varphi'(z^{(L)})$$

$$\delta^{(\ell)} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot \varphi'(z^{(\ell)})$$

Then gradients of params:

$$\partial L/\partial W^{(\ell)} = \delta^{(\ell)} a^{(\ell-1)\top}, \quad \partial L/\partial b^{(\ell)} = \delta^{(\ell)}$$



✦ Autograd

Modern frameworks (PyTorch, JAX, TF) build a computation graph and apply chain rule automatically. You write the forward pass; gradients come free.

★ *Watch out:* exploding gradients (→ clip), vanishing gradients (→ ReLU, residuals, normalization), inplace ops, retained graphs.

4. Loss & Optimizers

Common Losses

Regression

MSE: $(1/N)\ \Sigma\ (y - \hat{y})^2$

MAE: $(1/N)\ \Sigma\ |y - \hat{y}|$

Huber: smooth blend

Classification

Cross-entropy: $-\Sigma\ y \log \hat{p}$

Focal: down-weights easy ex.

Hinge: SVM-style

Other special purpose: triplet, contrastive, CTC, KL divergence, Wasserstein.

Gradient Descent Variants

- Batch GD — full dataset per step (slow, smooth)
- Stochastic (SGD) — 1 sample per step (noisy, fast)
- Mini-batch SGD — middle ground, the default

$$w \leftarrow w - \eta \cdot \nabla_w L(w;\ \text{mini-batch})$$

Momentum & Adaptive Methods

Momentum

$v \leftarrow \beta\ v + (1-\beta)\ \nabla L$

$w \leftarrow w - \eta\ v$

smooths zig-zags

RMSProp

$s \leftarrow \beta s + (1-\beta)(\nabla L)^2$

$w \leftarrow w - \eta\ \nabla L\ /\ \sqrt{s}$

per-param LR

Adam = Momentum + RMSProp

$$m \leftarrow \beta_1 m + (1-\beta_1)\ g, \quad v \leftarrow \beta_2 v + (1-\beta_2)\ g^2$$

$$\hat{m} = m/(1-\beta_1^t), \quad \hat{v} = v/(1-\beta_2^t)$$

$$w \leftarrow w - \eta \cdot \hat{m}\ /\ (\sqrt{\hat{v}} + \varepsilon)$$

default for ~everything. AdamW = Adam + decoupled weight decay.

Learning Rate Schedules

- step decay — halve every N epochs
- cosine annealing — smooth decrease
- warmup + cosine — standard for big Transformers
- one-cycle (Smith) — fast convergence

5. Regularization

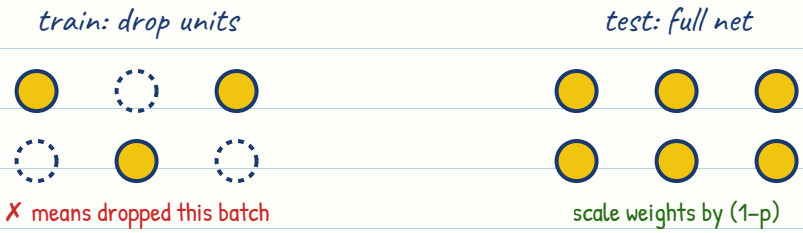
Big NNs overfit easily. Fight back!

Weight Penalties

- L2 (weight decay) — add $\lambda \Sigma w^2$ → shrink weights
- L1 — add $\lambda \Sigma |w|$ → sparsity

Dropout

During training, randomly zero each unit with prob p (≈ 0.5). At test time, use all units (with scaling).



Normalization

Stabilises training, lets you use bigger LRs.

- BatchNorm — normalize across batch dim per feature. Great for CNNs.
- LayerNorm — normalize across features per sample. Standard in Transformers.
- GroupNorm — between batch & layer; works with tiny batches.
- RMSNorm — like LayerNorm but skip the mean. Used in LLaMA.

$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}, \quad y = \gamma \hat{x} + \beta$

Other Tricks

- Early stopping — stop when val loss stops improving
- Data augmentation — flip, crop, jitter, mixup, cutmix
- Label smoothing — target = 0.9, not 1.0
- Stochastic depth — drop whole layers (ResNet)

★ Best regulariser of all: more data

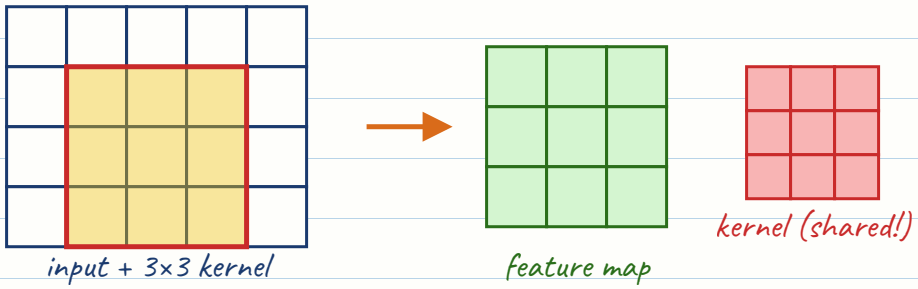
6. Convolutional NNs

Specialised for grid-like data (images, audio spectrograms). Key ideas: local connectivity, weight sharing, translation invariance.

The Convolution

Slide a small kernel (e.g. 3×3) over the input, computing dot products.

$$y[i, j] = \Sigma_{u, v}\, x[i+u, j+v] \cdot k[u, v]$$



Hyperparameters

- *kernel size* — usually 3×3 (sometimes 5, 7, 1)
- *stride* — step size; 2 → downsamples
- *padding* — "same" preserves H×W, "valid" shrinks
- *channels* — input C_{in} → output C_{out} feature maps

$$\text{output size} = \lfloor (H - K + 2P) / S \rfloor + 1$$

Pooling

Downsample feature maps. Max-pool 2×2 with stride 2 cuts H,W by half.

Provides shift-invariance; modern nets often replace it with strided convs.

7. CNN Architectures

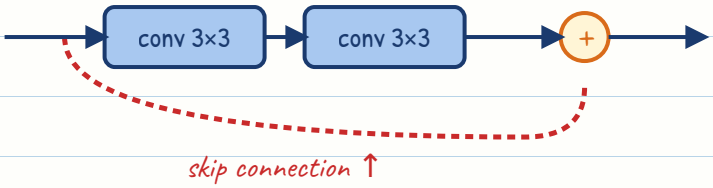
A Quick Tour

- LeNet-5 (1998) — digits, conv + pool + FC
- AlexNet (2012) — 8 layers, ReLU, dropout, GPU → kicked off deep learning
- VGG (2014) — stack of 3×3 convs, depth helps
- GoogLeNet / Inception (2014) — parallel branches w/ different kernel sizes
- ResNet (2015) — residual / skip connections, 152+ layers
- DenseNet (2016) — every layer connects to every later layer
- EfficientNet (2019) — compound scaling of depth × width × resolution
- ConvNeXt (2022) — modernised CNN that rivals ViTs

Residual Block — Why It Works

$$y = F(x, W) + x$$

Identity path lets gradients flow → trains 1000+ layer nets!



Vision Transformers (ViT)

Split image into patches (e.g. 16×16). Treat each patch as a token → feed into a vanilla Transformer encoder. Add a [CLS] token + positional embedding.

→ outperforms CNNs at large data scale.

Common Tasks

- Image classification — ImageNet
- Object detection — Faster R-CNN, YOLO, DETR
- Segmentation — U-Net, Mask R-CNN, SAM
- Style transfer, super-resolution, depth estimation

8. Sequence Models

For ordered data: language, audio, time series, video frames.

✦ RNN Recap

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

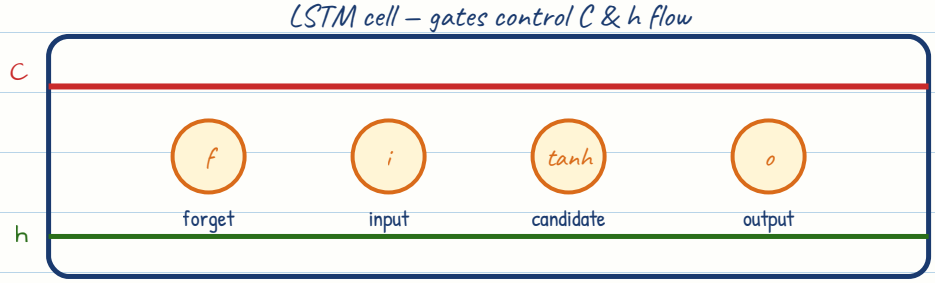
One set of weights, unrolled over time. Backprop through time (BPTT).

Issue: repeated multiplication by W → gradients vanish ($||W|| < 1$) or explode ($||W|| > 1$).

✦ LSTM

Add a cell state with multiplicative gates that protect gradient flow:

- *forget gate* — what to drop from cell
- *input gate* — what new info to store
- *output gate* — what part of cell to expose



✦ GRU

Merge cell + hidden state. Two gates (reset, update). Fewer params than LSTM, comparable performance.

✦ Bidirectional & Stacked

- *Bidirectional*: run one RNN forward, another backward; concat → full context
- *Stacked / deep*: feed $h^{(l-1)}$ into next RNN layer
- *Encoder-decoder (seq2seq)*: encode source, decode target

RNNs are still useful for streaming / very long sequences where Transformers are expensive, but Transformers dominate most tasks now.

9. Transformers (DL view)

Architecture-agnostic — works on text, images, audio, code, proteins, RL trajectories!

Scaled Dot-Product Attention

$$A = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

Each token's new representation = weighted average of all values.

Multi-Head + Residual + LN

- *Multi-head* — h parallel attention "channels"
- *Residual* — x + Sublayer(x), keeps gradients alive
- *LayerNorm* — pre-norm or post-norm
- *FFN* — 2-layer MLP with GELU between attention blocks

Why Transformers Scale

- parallel computation → maps perfectly to GPUs/TPUs
- same op on every token → easy to shard
- scaling laws — bigger model + more data ⇒ better loss reliably
- transfer learning — pretrain once, finetune many tasks

Efficient Attention Variants

Vanilla self-attention is $O(N^2)$. For long sequences:

- *Sparse* — Longformer, BigBird (windowed + global tokens)
- *Linear / kernel-based* — Performer, Linformer
- *FlashAttention* — exact, but I/O-aware → 2-4× faster ⚡
- *Mamba / SSM* — state-space models, recurrent & parallel

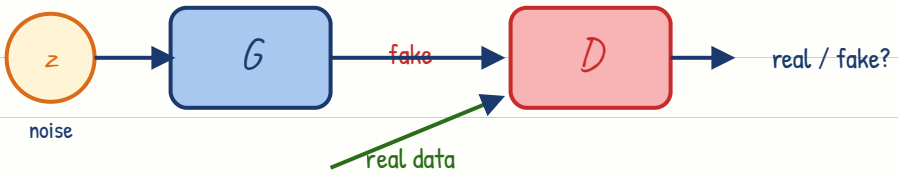
Beyond Text

- *Vision Transformer (ViT)* — patch tokens
- *Audio Transformer (wav2vec, Whisper)*
- *Multimodal (CLIP, Flamingo, LLaVA)* — joint text + image
- *Decision Transformer* — offline RL as sequence modelling

> *Big picture:* Transformer = "general-purpose differentiable computer". Most modern DL is Transformer variants + scale.

10. GANs

Two networks play a minimax game: Generator G tries to fake data, Discriminator D tries to spot fakes.



The Objective

$$\min_G \max_D \; E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))]$$

At equilibrium, G's distribution matches p_{data} and D outputs 0.5 everywhere.

Training Tricks

- Non-saturating loss: G maximises log D(G(z)) — better gradients
- Wasserstein GAN (WGAN-GP) — Earth Mover's distance, more stable
- Spectral norm — Lipschitz constraint on D
- Progressive growing / StyleGAN — high-res face generation

Variants

- cGAN — condition on label / text
- CycleGAN — unpaired image-to-image (horse ↔ zebra)
- Pix2Pix — paired conditional translation
- BigGAN, StyleGAN3 — SOTA image synthesis

✓ Pros

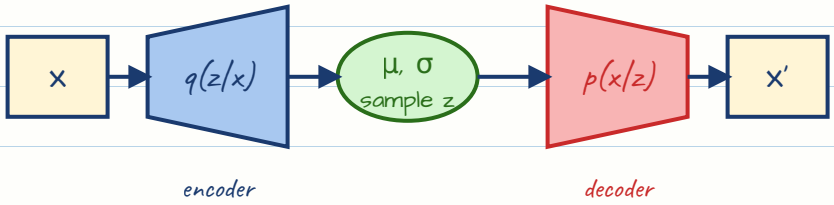
- sharp samples (esp. images)
- fast sampling — one forward pass
- no explicit likelihood

✗ Cons

- unstable training
- mode collapse
- hard to evaluate (FID, IS)

11. VAEs

Variational Autoencoder = probabilistic encoder + decoder + a prior on latents.



✦ ELBO (Evidence Lower Bound)

We can't compute log p(x) directly, so maximize a lower bound:

$$\text{ELBO} = E_{q(z|x)}[\log p(x|z)] - KL(q(z|x) \| p(z))$$

Two terms:

- Reconstruction — decoder recovers x from z
- KL regularizer — keep q close to prior $p(z) = \mathcal{N}(0, I)$

✦ Reparameterization Trick

Can't backprop through random sampling. Rewrite:

$$z = \mu(x) + \sigma(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

→ randomness is now in ε; gradients flow through μ, σ. □

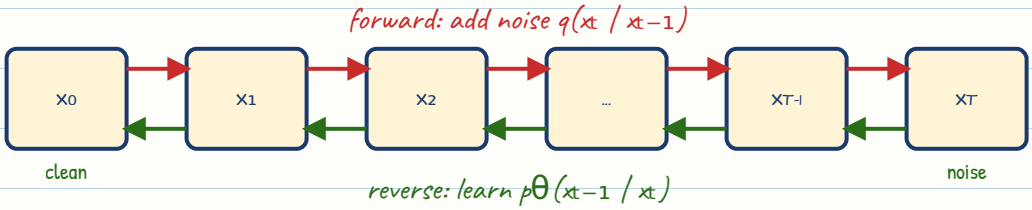
✦ Variants

- β-VAE — scale the KL term → disentangled latents
- VQ-VAE — quantize z to a codebook (used in DALL·E, Soundstream)
- Hierarchical VAE — multi-level latents (NVAE, VDVAE)

★ *vs GANs:* VAEs give a likelihood + smooth latent space but samples can be blurry. GANs give sharper images but no latent inference.

12. Diffusion Models

SOTA for image (and audio, video, 3D) generation. Idea: gradually add noise to data, learn to reverse the process.



✦ Forward Process (fixed)

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I)$$

After T steps, $x_T \approx$ pure Gaussian noise. Closed form:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1-\bar{\alpha}_t} \cdot \varepsilon$$

✦ Training Objective (DDPM)

Train a U-Net (with timestep conditioning) to predict the noise:

$$L = E_{t,\ x_0,\ \varepsilon} \left\| \varepsilon - \varepsilon_\theta(x_t, t) \right\|^2$$

That's it! Stable to train, no adversaries needed. □

✦ Sampling

Start from $x_T \sim \mathcal{N}(0, I)$. Iteratively denoise from t=T to t=1.

- DDIM — deterministic, fewer steps (~50)
- DPM-Solver — clever ODE solver, ~10 steps
- Classifier-free guidance — boost prompt adherence

✦ Stable Diffusion / Latent Diffusion

Run the diffusion in VAE-encoded latent space (e.g. 64×64 instead of 512×512). 10× faster, same quality. □

13. Training Tips & Tricks

Initialization

- Xavier / Glorot — for tanh / sigmoid: Var = $2/(n_{in}+n_{out})$
- He / Kaiming — for ReLU: Var = $2/n_{in}$
- Orthogonal — RNNs

Stable Training

- Gradient clipping — clip $||g||_2 \leq \tau$ (esp. RNNs, Transformers)
- Mixed precision — fp16/bf16 for speed + fp32 for stability
- Warmup — small LR for first 1-5% of steps
- Cosine schedule — decay LR smoothly to 0
- EMA of weights — smoother eval model

Hyperparameter Tuning

- most important: learning rate, batch size, model size
- random search > grid search
- Bayesian (Optuna, Ax) for expensive runs
- track everything with W&B / TensorBoard

Debugging a Network

1. overfit a tiny batch (5-10 examples) — should go to ~0 loss
2. check loss decreasing on full train set
3. compare to a baseline (linear, kNN)
4. visualise inputs, outputs, gradients, weights distributions
5. if NaN: lower LR, clip grads, check data

Scaling Up

- Data parallel — replicate model, split batch
- Model parallel — split model across GPUs
- Pipeline parallel — split layers + micro-batches
- ZeRO / FSDP — shard optimizer states, gradients, params
- Gradient checkpointing — recompute activations to save memory

★ Reproducibility: set random seeds, log code version, note CUDA version. Save checkpoints + hyperparams. Future-you will thank you. □

Big Picture

DL Building Blocks



Choosing an Architecture

- Tabular data → MLP, gradient boosting (still strong!)
- Images → CNN (ResNet, EfficientNet) or ViT
- Text → Transformer (pretrained: BERT, GPT, T5)
- Audio → CNN on spectrogram or wav2vec / Whisper
- Time series → LSTM, Temporal Conv, Transformer
- Graphs → GNNs (GCN, GAT, GraphSAGE)
- Generation → diffusion (images), LLMs (text)

Top 10 Tips

1. start simple, add complexity only when needed
2. pretrained models are your friend □
3. normalize inputs & outputs
4. watch the loss curves; don't just chase accuracy
5. look at predictions, not just metrics
6. fix data quality before architecture tweaks
7. save checkpoints often
8. use mixed precision for speed
9. profile before optimising
10. have fun, break things, learn □

— end of DL notes —